20 PAGES

Terraform

in Plain English

FROM ZERO TO REAL INFRASTRUCTURE

- PRACTICAL EXAMPLES
- WORKSHOP STYLE
- REAL CLOUD SCENARIOS
- BEGINNER FRIENDLY



- WRITE INFRASTRUCTURE AS CODE
- DEPLOY MANAGE UPDATE
- TROUBLESHOOT LIKE AN ENGINEER
- BUILD SKILLS FOR REAL-WORLD DEVOPS JOBS

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Terraform Explained Without the Jargon

01

1. What is Terraform?

- Terraform is an open-source tool used to create, manage and update infrastructure as code (IaC).
- You write configuration files that describe the infrastructure you want.
- Terraform then communicates with cloud providers (like AWS, Azure or Google Cloud) to create and manage the actual resources for you.

2. What problem does it solve?

- Without Terraform, you create infrastructure manually in the cloud console or with scripts.
- Manual setup is slow, error-prone and difficult to repeat.
- Terraform solves this by letting you define infrastructure in code, so you can automate, version, and safely recreate the same environment every time.

3. Infrastructure as Code (IaC) in simple terms

- Infrastructure as Code (IaC) means you manage servers, networks, storage and other cloud resources using code instead of clicking in aconsole.
- Instead of manually creating an EC2 server, a VPC and a database, you write a Terraform configuration file that describes all of them.
- You can then recreate the same infrastructure in minutes, in any environment (dev, staging, prod).

Example: A real cloud scenario

You want to create a web application environment with:

- A Virtual Private Cloud (VPC)
- A public subnet
- An EC2 server
- A security group
- An S3 bucket for files

Instead of clicking each service in the AWS console, you write one Terraform configuration and deploy all of them together.

4. Manual vs Terraform-managed infrastructure

Manual infrastructure (Cloud console)	Terraform-managed infrastructure
✗ You click and configure each resource by hand.	✓ You define everything in code.
✗ Easy to make mistakes.	✓ Repeatable and consistent.
✗ Difficult to repeat the same setup.	✓ Changes are versioned and reviewed.
✗ Hard to track changes.	✓ Easier to manage across environments.
✗ Not suitable for team collaboration.	✓ Works well with Git and CI/CD.

5. How Terraform fits into a DevOps engineer's daily work

- Create and manage cloud infrastructure.
- Automate environment setup (dev, staging, prod).
- Work with version control (Git).
- Integrate with CI/CD pipelines.
- Collaborate with developers and other engineers.
- Make infrastructure changes safe, repeatable and auditable.

Why DevOps engineers use it

- Speeds up environment provisioning.
- Reduces manual work and human error.
- Supports automation and scalability.
- Helps maintain consistent infrastructure.
- Fits into modern DevOps workflows.

6. How it works (high level view)



Engineer (writes Terraform configuration) → **Terraform** (reads config and plans changes) → **Cloud Provider** (AWS, Azure, Google Cloud) (API) → **Infrastructure** (actual cloud resources created and managed)

How Terraform Actually Works

1. The main parts of Terraform

Configuration	• You write .tf files that describe the infrastructure you want. • This is the desired state.
Provider	• A plugin that communicates with cloud providers (AWS, Azure, Google Cloud). • It translates your configuration into API calls.
State	• A file that keeps track of the resources Terraform manages. • It helps Terraform know what already exists.
Resources	• The actual infrastructure objects you create (e.g. EC2, S3, VPC, database). • Defined in your configuration using resource blocks.

2. Why Terraform needs a provider?

- Terraform itself does not create cloud resources directly.
- It uses a provider to communicate with the cloud provider's API (e.g. AWS API).
- The provider takes your configuration and sends the correct API requests to create, update or delete resources.
- Different cloud providers have different APIs, so Terraform uses a provider for each one (AWS, Azure, Google Cloud, etc).

Example: AWS Provider

- You configure the AWS provider with your credentials and region.
- Terraform then uses this provider to create and manage AWS resources on your behalf.

3. How Terraform compares what you want with what it manages

Your Configuration (desired state)

```
resource "aws_s3_bucket"
"veriqta_bucket" {
  bucket = "veriqta-demo"
  tags = {
    Name = "Veriqta"
  }
}
```

Terraform Plan (compare)

- Terraform checks your configuration against the current state.
- It figures out what needs to be created, updated or deleted.

Provider API (execute changes)

- The provider sends the required API requests to the cloud provider (e.g. AWS).
- This creates, updates or deletes resources.

Cloud Resources (real infrastructure)

EC2 S3 VPC

4. Desired configuration vs real infrastructure

Desired configuration (what you want)	Real infrastructure (what actually exists)
• Defined in your .tf files. • Describes the infrastructure state you want. • Stored in version control (Git). • Can be reused and shared. • Example: an S3 bucket with specific settings.	• The actual resources in the cloud (AWS, Azure, GCP). • Managed by Terraform using a provider. • Can change over time. • Terraform keeps track of this in the state file. • Example: the S3 bucket that exists in your AWS account.

5. Key takeaways

- You write the desired state in configuration files.
- Terraform compares this with the real infrastructure using the state.
- A provider communicates with the cloud provider's API.
- Terraform shows you a plan of changes before applying them.
- When you apply, the provider creates, updates or deletes the resources.

Remember This

- Terraform does not automatically detect everything outside of its management.
- The state file is important. Do not delete it or commit it to GitHub.

Common Beginner Mistake

- Thinking Terraform creates resources without a provider.
- Assuming the real infrastructure always matches your configuration.

Your First Terraform Project

03

1. Install Terraform and verify the installation

- Download Terraform from the official website or use a package manager.
- After installation, verify it is working.
- You should see the installed version.

Terraform

```
terraform version
```

```
Terraform v1.6.0
on linux_amd64
+ provider registry.terraform.io/hashicorp/...
```

2. Create a project directory and a main.tf file

- Create a folder for your project.
- Move into the folder.
- Create a file named main.tf.
- This file will contain your Terraform configuration.

```
mkdir terraform-demo
cd terraform-demo
touch main.tf
ls
```

```
terraform-demo/
└── main.tf
```

3. What do .tf files contain?

- .tf files contain Terraform configuration written in HashiCorp Configuration Language (HCL).
- They define the providers, resources, variables, outputs and other settings.
- Terraform reads these files to understand what infrastructure you want.

Common .tf files

main.tf	Main configuration (resources)
variables.tf	Input variables
outputs.tf	Output values
providers.tf	Provider settings

4. Write a simple configuration using terraform_data

main.tf

```
# Simple Terraform configuration
terraform {
  required_version = ">= 1.5.0"
}

resource "terraform_data" "example" {
  input = "Hello from Terraform!"
}
```

Explanation

- terraform block: Specifies the required Terraform version.
- terraform_data: A built-in resource used for learning and testing.
- input: A simple value we give to this resource.
- This does not create a real cloud resource. It is used to understand how Terraform works.

5. Practice: Run terraform init and terraform validate

Step 1: Initialize the project

```
terraform init
```

- Downloads the required providers.
- Creates a .terraform directory.
- Prepares the working directory.

```
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!
```

Step 2: Validate the configuration

```
terraform validate
```

- Checks if your configuration is syntactically correct.
- Does not create any infrastructure.
- Helps catch mistakes early.

```
Success! The configuration is valid.
```

Next step

After successful validation, you can run terraform plan to see what Terraform will do.

Common beginner mistake

- Running terraform apply without checking for errors.
- Always run terraform validate (and plan) first.

The Five Terraform Commands You Must Understand

1. terraform init

Prepare the working directory

```
terraform init
```

- Downloads the required provider plugins.
- Sets up the .terraform directory.
- Initializes the backend (if using remote state).
- Must be run before plan or apply.





.terraform/

2. terraform fmt

Format configuration files

```
terraform fmt
```

- Formats your .tf files to a consistent style.
- Makes your code easier to read.
- Does not change the logic or behaviour of your configuration.
- Good practice to run before committing to Git.



3. terraform validate

Check configuration validity

```
terraform validate
```

- Checks if your configuration is syntactically correct.
- Verifies required arguments and block structure.
- Does not contact the cloud provider.
- Catches common mistakes early.



4. terraform plan

Preview proposed changes

```
terraform plan
```

- Compares your configuration (desired state) with the current state (actual state).
- Shows what Terraform will create, update or delete.
- Does not make any changes to your infrastructure.
- Always review the plan before applying.



5. terraform apply

Make approved changes

```
terraform apply
```

- Applies the changes shown in the plan.
- Creates, updates or deletes resources in your cloud environment.
- You will be asked to confirm (type yes) unless you use -auto-approve.
- Always review the plan before applying.



Apply changes to real infrastructure

What each command does

Command	Purpose	Changes infrastructure?
terraform init	Prepare working directory	No
terraform fmt	Format .tf files	No
terraform validate	Check configuration	No
terraform plan	Preview proposed changes	No
terraform apply	Create, update or delete resources	Yes

Important: Validation does not guarantee a successful deployment

- terraform validate only checks syntax and configuration structure.
- It does not verify:
  - Your cloud provider credentials.
  - Your permissions (IAM roles).
  - Whether the resources can be created.
  - Quotas or limits in your cloud account.
  - Network or service dependencies.
- You can have a valid configuration, but terraform apply can still fail.

Terraform Language Made Simple

05

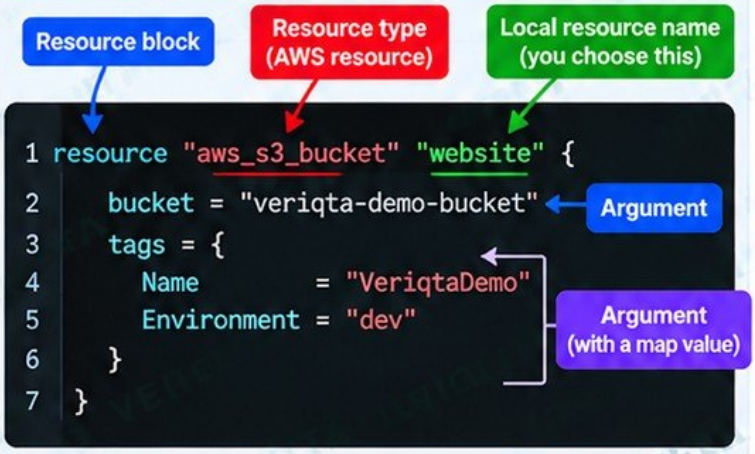
1. What is the Terraform language?

- Terraform uses a language called HCL (HashiCorp Configuration Language).
- HCL lets you define infrastructure in simple, human-readable files with the .tf extension.
- You describe what you want (resources, settings, values) using blocks, arguments and expressions.
- Terraform reads these files and uses a provider to create, update or delete the resources for you.

2. Main building blocks

Blocks	Group related configuration together (e.g. resource, variable, provider).
Arguments	Settings inside a block (e.g. name = "value").
Expressions	Used to reference values, perform simple logic or combine strings (e.g. var.environment, "web-${var.env}").

3. How a resource block is structured



4. Resource type vs local resource name

Resource type

- Tells Terraform which resource to manage.
- Example: aws_s3_bucket, aws_instance, aws_vpc.
- Each type is provided by a cloud provider (e.g. AWS).

Local resource name

- A name you choose inside your configuration.
- Used to refer to this resource within your .tf files (e.g. aws_s3_bucket.website).
- Does not have to be the same as the actual name in the cloud.

5. A small working example (line by line)

```
1 resource "aws_s3_bucket" "website" {
2   bucket = "veriqta-demo-bucket"
3   tags = {
4     Name        = "VeriqtaDemo"
5     Environment = "dev"
6   }
7 }
```

1. Starts a resource block.
2. Resource type: AWS S3 bucket.
3. Local name: website (you choose this).
4. Bucket name in AWS (must be globally unique).
5. Starts a map of tags.
6. A tag key and value.
7. Another tag.
8. Ends the tags block.
9. Ends the resource block.

6. Beginner mistake

Confusing a Terraform resource name with the cloud resource's actual name.

- The local name (e.g. website) is only used inside Terraform files. It is not the name of the S3 bucket in AWS.
- The actual bucket name is the value of the bucket argument (e.g. veriqta-demo-bucket).

7. Key takeaways

- Terraform uses HCL with .tf files.
- Blocks, arguments and expressions define your infrastructure.
- Resource type comes from the provider.
- Local resource name is your own label.
- Always understand what each argument does.

Variables: Stop Hardcoding Everything

06

1. What are input variables and why do they exist?

- Input variables let you pass values into your Terraform configuration instead of hardcoding them.
- They make your configuration reusable, flexible and easier to manage.
- You can use different values for different environments (dev, staging, prod).
- They separate configuration from code, so you don't have to edit files every time.

2. variable block, type, default and description

```
variable "environment" {
  type =  string
  default = "dev"
  description = "The
  environment name
  (dev, staging, prod)."
}
```

- Variable name (you use var.environment)
- Data type (string, number, bool, etc.)
- Default value (used if no value is provided)
- Description (explains what the variable is for)

3. Using var.environment in a configuration

```
resource "aws_s3_bucket" "website" {
  bucket = "veriqta-${var.environment}-bucket"
  tags = {
    Environment = var.environment
    Name        = "veriqta-demo"
  }
}
```

Using the variable in the configuration

What happens?

- If var.environment is dev, the bucket will be veriqta-dev-bucket.
- You can use the same configuration for staging and prod by changing only the variable value.

4. .tfvars files and variable overrides

- A .tfvars file contains values for your variables.
- It keeps your configuration clean and organized.
- You can have different .tfvars files for different environments.
- You can also override variables using the -var option on the command line.

Example: dev.tfvars

```
environment = "dev"
bucket_name = "veriqta-demo"
region      = "us-east-1"
```

Example: prod.tfvars

```
environment = "prod"
bucket_name = "veriqta-prod"
region      = "us-east-1"
```

Overriding variables from the command line

```
terraform plan -var="environment=staging" \
               -var="bucket_name=veriqta-staging"
```

5. Example: Complete working setup

variables.tf

```
variable "environment" {
  type        = string
  default     = "dev"
  description = "Environment
  name (dev, staging, prod)."
}
```

main.tf

```
resource "aws_s3_bucket" "website" {
  bucket = "veriqta-${var.environment}-
  bucket"
  tags = {
    Environment = var.environment
    Project     = "veriqta-demo"
  }
}
```

dev.tfvars

```
environment = "dev"
bucket_name = "veriqta-demo"
region      = "us-east-1"
```

Run the commands

```
terraform init
terraform validate
terraform plan -var-file="dev.tfvars"
```

6. Security reminder

- Marking a variable as sensitive = true hides the value in CLI output, but it does NOT prevent the value from being stored in the state file.
- Sensitive data (e.g. passwords, API keys) can still end up in the state file.
- Do not store secrets directly in Terraform variables.
- Use a secrets manager (e.g. AWS Secrets Manager) for sensitive information.

7. Key takeaways

- Use input variables to avoid hardcoding values.
- Understand type, default and description.
- Use var.environment (or any variable) in your configuration.
- Use .tfvars files for different environments.
- You can override variables from the command line.
- Sensitive variables do not automatically prevent secrets from entering the state file.

Outputs and Locals: Make Terraform Easier to Use

07

1. What are outputs and locals?

- Outputs let you display useful information about the infrastructure you create.
- Locals are reusable expressions that help you avoid repeating values.
- They make your configuration cleaner, easier to maintain and more consistent.
- Variables, locals and outputs each have a different purpose.

2. Variables vs Locals vs Outputs

Feature	Variables	Locals	Outputs
Purpose	Input values from outside	Reusable expressions inside config	Display values after apply
Defined with	variable	locals	output
Value comes from	User or .tfvars	Expression in your config	Resource attributes or local values
Visible to others	Yes (input)	No (internal)	Yes (after apply)
Example	var.environment	local.project_name	output.bucket_name

3. Using locals for a naming convention

```
locals {
  project_name = "veriqta"
  environment  = "dev"
  name_prefix = "${local.project_name}-${local.environment}"
  common_tags = {
    Project     = local.project_name
    Environment = local.environment
  }
}
```

Explanation

- local.project_name: A reusable value.
- local.environment: The environment name.
- local.name_prefix: Combines values using an expression.
- local.common_tags: A map of tags used by multiple resources.

4. Using output to display useful information

```
output "bucket_name" {
  description = "Name of the S3 bucket"
  value       = aws_s3_bucket.website.bucket
}

output "bucket_arn" {
  description = "ARN of the S3 bucket"
  value       = aws_s3_bucket.website.arn
}
```

Explanation

- output "bucket_name": Displays the bucket name after apply.
- output "bucket_arn": Displays the bucket ARN.
- Outputs are useful for other modules, scripts or for sharing important information.

5. Complete example

```
locals {
  project_name = "veriqta"
  environment  = "dev"
  bucket_name  = "${local.project_name}-
                  ${local.environment}-bucket"
}
resource "aws_s3_bucket" "webite" {
  bucket = local.bucket_name
  tags   = local.common_tags
}
output "bucket_name" {
  value = aws_s3_bucket.website.bucket
}
```

6. Running terraform apply

```
$ terraform apply
...
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
bucket_name = "veriqta-dev-bucket"
bucket_arn  = "arn:aws:s3:::veriqta-dev-bucket"
```

7. Sensitive outputs require careful handling

```
output "db_password" {
  value     = aws_db_instance.db.password
  sensitive = true
}
```

Important

- Marking an output as sensitive hides the value in CLI output.
- It does NOT prevent the value from being stored in the state file.
- Sensitive data can still be exposed if the state file is not protected.
- Use a secrets manager for real secrets (e.g. AWS Secrets Manager).

8. Practice: Create a local naming convention and expose an output

- Create a local.project_name and local.environment.
- Use local.name_prefix to build a bucket name.
- Create an S3 bucket using this name.
- Add an output to display the bucket name.
- Run teraform apply and verify the output.

9. Key takeaways

- Outputs display useful information after apply.
- Locals help you reuse values and keep your code clean.
- Variables, locals and outputs have different purposes. Use locals for naming conventions and tags.
- Be careful with sensitive outputs. They do not prevent secrets from entering the state file.

Terraform Providers and Authentication

08

1. What are providers and what do they do?

- Providers allow Terraform to communicate with cloud platforms and other services (e.g. AWS, Azure, Google Cloud).
- They translate your Terraform configuration into API calls to create, update and delete resources.
- Terraform downloads the required provider plugins from the Terraform Registry.

Terraform

aws

Azure

Google Cloud

2. required_providers and provider versions

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

Provider name (local name)

Source on Terraform Registry

Version constraint (allows 5.x)

- required_providers tells Terraform which providers your configuration needs.
- source = "hashicorp/aws" means download from Terraform Registry.
- version = "~> 5.0" allows any 5.x version.
- Terraform saves exact versions in .terraform.lock.hcl to ensure consistent setups.

3. The .terraform.lock.hcl file

```
# .terraform.lock.hcl (simplified)

provider "registry.terraform.io/hashicorp/aws" {
  version     = "5.31.0"
  constraints = "~> 5.0"
  hashes = [
    "h1:abcdef...",
    "zh:123456..."
  ]
}
```

Key points

- Created automatically after terraform init.
- Locks the exact provider versions and checksums.
- Helps ensure everyone uses the same provider version.
- Commit this file to Git.

4. How AWS credentials are supplied

- Terraform uses the AWS provider, which follows the standard AWS credential chain.
- Common ways to supply credentials:

aws

1. Environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY)
2. AWS CLI configuration (~/.aws/credentials)
3. IAM roles (e.g. EC2, GitHub Actions OIDC)
4. Named profiles (provider "aws" { profile = "dev" })
5. IAM Identity Center (SSO)

5. Example: AWS provider configuration

```
provider "aws" {
  region = "us-east-1"

# Uses credentials from environment variables,
# AWS CLI, IAM role or named profile.
# No hardcoded access keys.
}
```

Explanation

- Specifies the AWS region.
- Credentials are supplied securely using one of the supported methods.
- Do not hardcode access keys in your configuration.

6. What NOT to do (hardcoded keys)

```
provider "aws" {
  region     = "us-east-1"
  access_key = "AKIA..."
  secret_key = "abcd1234..."
}
```

- Hardcoded access keys can be exposed in GitHub, logs, state files and backups.
- They can be accidentally shared and misused.
- Always use secure methods (environment variables, IAM roles, or a secrets manager).

7. Common failure: Authentication or authorization errors

```
Error: refreshing Terraform state: retrieving AWS
account ID: operation error STS: GetCallerIdentity,
https response error StatusCode: 403, RequestID: ...,
api error AccessDenied: User is not authorized
to perform: sts:GetCallerIdentity
```

Common causes

- Invalid or expired credentials.
- User does not have required IAM permissions.
- Wrong AWS region.
- Using the wrong profile.
- Account or role not configured correctly.

8. Key takeaway

- Providers let Terraform communicate with cloud platforms.
- Use required_providers and version constraints.
- Keep .terraform.lock.hcl in your repository.
- Use secure methods to supply AWS credentials.
- Never hardcode access keys.
- Authentication and authorization errors are common – read the error message carefully.

Your First AWS Resource with Terraform

09

1. What is an Amazon S3 bucket?

- Amazon S3 (Simple Storage Service) is a cloud storage service from AWS.
- It stores files, backups, logs, static websites and many other types of data.
- Each bucket has a globally unique name across all of AWS.
- In this example, you will create an S3 bucket using Terraform.



2. Terraform configuration for an S3 bucket

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "website" {
  bucket = "veriqta-demo-bucket-12345"
  tags = {
    Name        = "veriqta-demo"
    Environment = "dev"
  }
}
```

- AWS provider and region
- Resource type (aws_s3_bucket)
- Unique bucket name (must be globally unique)
- Tags (optional)

3. Run terraform plan before making changes

```
$ terraform plan

Terraform will perform the following actions:
  + create
    # aws_s3_bucket.website will be created
    + resource "aws_s3_bucket" "website" {
        bucket = "veriqta-demo-bucket-12345"
        ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

What this does

- Shows you what Terraform will create, update or delete.
- Helps you catch mistakes before making changes.
- No resources are created during terraform plan.

4. Apply the configuration

```
$ terraform apply
Terraform will perform the following actions:
  + create
    # aws_s3_bucket.website will be created
  ...
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.
  Enter a value: yes
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

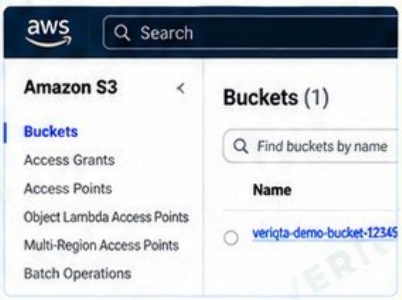
5. Verify the bucket exists

Option 1: Using AWS CLI

```
$ aws s3 ls
2026-10-15 10:24:33 veriqta-demo-bucket-12345
```

Option 2: Using AWS Console

- Log in to the AWS Console.
- Go to S3.
- You should see your bucket in the list.



6. Production awareness

Understand cloud costs

- S3 storage, requests and data transfer can incur charges.
- Clean up resources you no longer need.

Enable default encryption

- Use server-side encryption (SSE-S3 or SSE-KMS) to protect your data.

Block public access

- Keep public access blocked by default.
- Only allow public access when you have a specific and valid reason.

Cleanup when done

- Delete the bucket if it is only for learning.
- Avoid leaving unused resources running.

7. Example with security and best practices

```
provider "aws" {
  region = "us-east-1"
}
resource "aws_s3_bucket" "website" {
  bucket = "veriqta-demo-bucket-12345"
  tags = {
    Name        = "veriqta-demo"
    Environment = "dev"
  }
  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "AES256"
      }
    }
  }
  public_access_block {
    block_public_acls       = true
    block_public_policy     = true
    ignore_public_acls      = true
    restrict_public_buckets = true
  }
}
```

What's different here?

- Enables server-side encryption (AES256).
- Blocks public access to the bucket.
- Uses tags for better organization.
- Follows security best practices.

8. Common mistakes

- Bucket name already exists (must be globally unique).
- Authentication or authorization errors (wrong credentials or insufficient permissions).
- Forgetting to enable encryption.
- Accidentally allowing public access.
- Leaving resources running and incurring unnecessary costs.

9. Key takeaway

- You can create an AWS S3 bucket using Terraform.
- Always run terraform plan before applying.
- Consider security, costs and cleanup when working with real cloud resources.

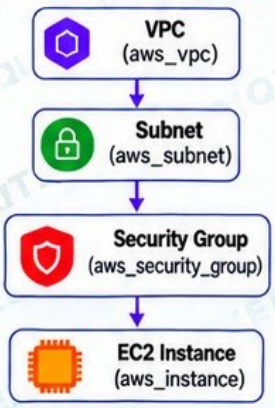
How Terraform Knows What to Change

10

1. Terraform's resource dependency graph

- Terraform builds a dependency graph of all resources in your configuration.
- It figures out the correct order to create, update or delete resources based on their relationships.
- This ensures resources are created in the right order and destroyed in the right order.
- You usually don't need to manage the graph manually because Terraform does this automatically.

Example dependency graph



2. Implicit dependencies (using references)

- When one resource references an attribute from another resource, Terraform automatically knows it depends on it.
- For example, an EC2 instance that uses a subnet ID will be created after the subnet.

```
resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.public.id # reference
  vpc_security_group_ids = [
    aws_security_group.web.id # reference
  ]
}
```

These references create implicit dependencies.

3. Using depends_on for explicit dependencies

- Sometimes Terraform cannot automatically detect a dependency (for example, when using provisioners, or when a resource relies on something outside of a direct reference).
- In such cases, you can use depends_on to explicitly tell Terraform which resource to wait for.

```
resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t3.micro"

  depends_on = [
    aws_s3_bucket.logs
  ]
}
```

When to use depends_on?

- When using provisioners.
- When a resource depends on something not referenced directly in the configuration.
- When working with modules and external systems.

Use depends_on only when necessary.
In most cases, Terraform automatically handles dependencies through resource references.

4. Understanding a plan

terrraform plan shows what Terraform will do by comparing your configuration with the current state.

```
$ terraform plan
...
  # aws_s3_bucket_logs will be created
  + resource "aws_s3_bucket" "logs" {
      bucket = "veriqta-logs-123"
    }

  # aws_instance.web will be updated in-place
  ~ resource "aws_instance" "web" {
      instance_type = "t3.micro" -> "t3.small"
    }

  # aws_security_group.db must be replaced
-/+ resource "aws_security_group" "db" {
      name = "db-sg" -> "db-sg-new" # forces replacement
```

5. Plan actions explained

Symbol	Action	Meaning	Example
+	Create	Resource does not exist and will be created.	+ aws_s3_bucket.logs
~	Update (in-place)	Resource exists and will be updated without destroying.	~ aws_instance.web instance_type = "t3.micro" -> "t3.small"
-/+	Replace	Resource must be destroyed and created again (cannot be updated in-place).	-/+ aws_security_group.db name = "db-sg" -> "db-sg-new"
–	Destroy	Resource will be deleted.	– aws_s3_bucket.old

6. Why replacement happens?

- Some changes cannot be made in-place due to cloud provider limitations.
- Examples:
  - Changing the name of a resource (e.g. S3 bucket).
  - Changing certain immutable fields (e.g. VPC CIDR).
  - Some instance settings.
- Terraform will destroy the old resource and create a new one when required.

7. Beginner mistake

Assuming Terraform always updates resources without replacing them.

- Some changes force a replacement, not an update.
- Always check the plan output carefully.
- Understand the impact of replacement on your applications and data.

8. Key takeaway

- Terraform builds a resource dependency graph automatically.
- Resource references create implicit dependencies.
- Use depends_on only when necessary.
- Always read the terraform plan output to understand create, update and replace actions.
- Not all changes are in-place updates — some will replace the resource.

Terraform State: The File You Must Protect

1. What is terraform.tfstate?

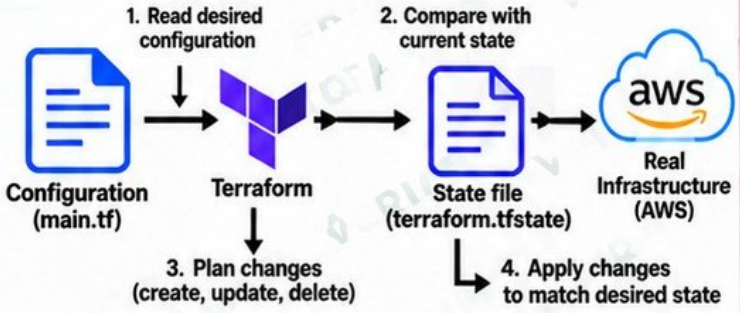
- **terraform.tfstate** is a JSON file that records information about the infrastructure Terraform manages.
- It contains details about resources, attributes, dependencies and metadata.
- It is created and updated automatically when you run **terraform apply**.

Example (simplified)

```
{
  "version": 4,
  "resources": [
    {
      "type": "aws_s3_bucket",
      "name": "website",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "attributes": {
            "id": "veriqta-demo-bucket-12345",
            "arn": "arn:aws:s3:::veriqta-demo-bucket-12345",
            "bucket": "veriqta-demo-bucket-12345",
            "region": "us-east-1"
          }
        }
      ]
    }
  ]
}
```

2. Why Terraform needs state

- Terraform uses the state file to keep track of what it has already created.
- It compares your configuration with the state to decide what to create, update or delete.
- Without state, Terraform cannot reliably know what resources exist or their current attributes.



3. What happens when state is lost, outdated or corrupted?

State is lost

- Terraform no longer knows what resources it created.
- It may try to create duplicate resources.
- You may have to manually recover or import resources.

State is outdated

- The state does not match the real infrastructure.
- Terraform may plan unexpected changes.
- It can attempt to recreate, modify or delete resources depending on the difference.

State is corrupted

- Terraform may fail to run.
- You may get errors during plan or apply.
- You might need to restore from a backup or use state recovery commands.

4. State may contain sensitive information

- The state file can contain credentials, tokens, passwords and other sensitive values.
- This can happen when resources or providers store sensitive data in the state.
- Even if a value is marked as sensitive, it can still be stored in the state file.

Example (sensitive data in state - simplified)

```
"attributes": {
  "access_key": "AKIAIOSFODNN7EXAMPLE",
  "secret_key": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLE",
  "db_password": "SuperSecret123!"
}
```

The state file is a real file with real data. Treat it as highly sensitive.

5. Never commit state files to GitHub



- Do NOT commit `terraform.tfstate` files to GitHub or any public repository.
- Anyone can read the file and access sensitive information.
- Always add state files to `.gitignore`.
- Use remote state (e.g. S3 + DynamoDB) for real projects.

.gitignore (example)

```
# Terraform files
.terraform/
terraform.tfstate
terraform.tfstate.*
crash.log
*.tfstate
*.tfstate.backup
```

Keep state files out of GitHub

6. Best practices for state security

- Use remote state storage (e.g. AWS S3) with encryption.
- Enable state locking (e.g. DynamoDB) to prevent concurrent changes.
- Restrict access to the state file using IAM.
- Do not store secrets directly in Terraform configuration when possible.
- Regularly back up your state.
- Treat the state file as sensitive data.

7. Key takeaway

- The state file records what Terraform manages.
- Terraform needs state to track and compare infrastructure.
- State can be lost, outdated or corrupted, causing problems.
- State can contain sensitive information.
- Never commit state files to GitHub.
- Use secure remote state storage for real projects.
- Always follow state security best practices.

Remote State and State Locking Explained

1. Why local state is unsuitable for most teams?

- Local state is stored on your computer in terraform.tfstate.
- It is not shared with other team members.
- It can easily be lost, overwritten or become out of sync.
- It does not provide locking, which can lead to conflicting changes.
- It is suitable for simple learning and testing, but not recommended for team or production environments.



Local state is only recommended for personal learning and temporary testing.

2. Remote backend using Amazon S3

- A remote backend stores the state file in an S3 bucket.
- All team members work with the same state file.
- Terraform automatically downloads and updates the state in the backend.
- You configure the backend in your Terraform code.

```
terraform {
  backend "s3" {
    bucket         = "veriqta-terraform-state"
    key            = "dev/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "veriqta-terraform-locks"
  }
}
```

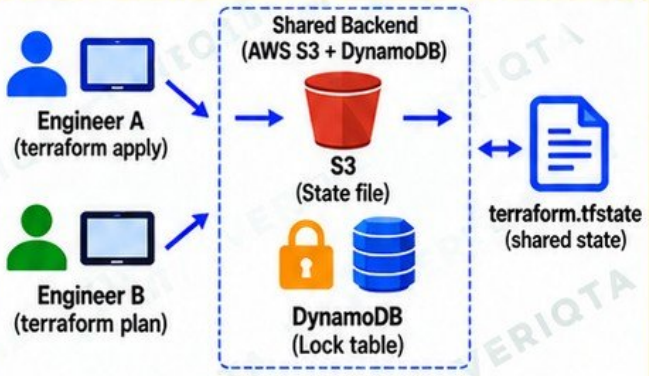
Explanation

- `bucket`: S3 bucket name.
- `key`: path to the state file.
- `region`: AWS region.
- `encrypt`: encrypts the state file.
- `dynamodb_table`: used for state locking.

3. How state locking works

- State locking prevents multiple people from making changes to the state at the same time.
- Terraform uses a locking mechanism (e.g. DynamoDB with Amazon S3) to ensure only one operation can modify the state at a time.
- If Engineer A is running terraform apply, Engineer B must wait until the lock is released.

Diagram: Team workflow with remote state and locking



4. S3 bucket configuration (example)

```
resource "aws_s3_bucket" "terraform_state" {
  bucket = "veriqta-terraform-state"
}
resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = aws_s3_bucket.terraform_state.id
  versioning_configuration {
    status = "Enabled"
  }
}
resource "aws_s3_bucket_server_side_encryption_configuration"
 "state_encryption" {
  bucket = aws_s3_bucket.terraform_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

Explanation

- Creates an S3 bucket for the state file.
- Enables versioning to keep previous state file versions.
- Enables server-side encryption (AES256) for data at rest.

5. DynamoDB table for state locking (example)

```
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "veriqta-terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

Explanation

- Creates a DynamoDB table for state locking.
- The LockID attribute is used by Terraform to manage locks.
- This prevents multiple engineers from changing the state at the same time.

6. Key features and security benefits

Encryption

- Encrypts the state file at rest using S3 encryption (e.g. AES256).
- Protects sensitive data such as credentials.

Access control

- Use IAM policies to restrict who can read or write the state file.
- Follow the principle of least privilege.

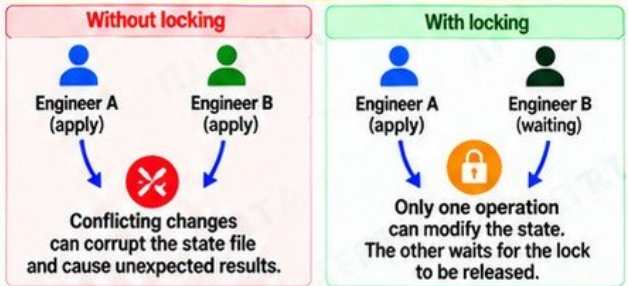
Versioning

- Keeps previous versions of the state file.
- Helps with recovery if the state becomes corrupted or deleted.

State locking

- DynamoDB prevents concurrent changes.
- Only one operation can modify the state at a time.
- Reduces the risk of state corruption.

7. What happens without locking?



8. Key takeaway

- Local state is not suitable for team or production environments.
- Use a remote backend (e.g. Amazon S3) to store the state file.
- Enable state locking (e.g. DynamoDB) to prevent conflicting changes.
- Use encryption, access controls and versioning to protect the state file.
- All team members should work with the same remote state.
- Remote state and locking make your infrastructure management safer, more reliable and more collaborative.

Terraform Modules: Reuse Infrastructure Code

1. What is a Terraform module?

- A module is a reusable collection of Terraform resources.
- It allows you to organize infrastructure code, reduce duplication and reuse the same code in different environments.
- Modules make your code cleaner, easier to maintain and more consistent.



Terraform Modules

2. Root module vs child module

Root module

- The main module you run from your working directory.
- Calls one or more child modules.
- Contains your environment-specific configuration (e.g. dev, staging, prod).

Child module

- A reusable module that contains resources.
- Can be stored locally or in a remote repository.
- Receives input variables and returns outputs.
- Can be used by multiple root modules.

3. Example module folder structure

```
project/
├── main.tf          # Root module
├── variables.tf     # Root variables
├── outputs.tf       # Root outputs
└── modules/
    └── s3-bucket/       # Child module
        ├── main.tf      # Resources
        ├── variables.tf # Input variables
        └── outputs.tf   # Output values
```

Key points

- The root module calls the s3-bucket module in the `modules/` folder.
- The child module contains the actual resources.
- This structure can be reused for other modules (e.g. VPC, EC2, RDS).

4. Child module example (modules/s3-bucket/main.tf)

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  tags = {
    Name        = var.bucket_name
    Environment = var.environment
  }
}
```

Uses input variables

- var.bucket_name and var.environment are passed from the root module.
- The child module focuses only on reusable resources.

5. Root module calling the child module (main.tf)

```
module "s3_bucket" {
  source      = "./modules/s3-bucket" # local path
  bucket_name = "veriqta-demo-bucket" # input variable
  environment = "dev"
}
```

What this does

- Calls the s3-bucket module from the `modules/` folder.
- Passes values into the module using input variables.
- The same module can be reused with different values in other environments.

6. Module inputs and outputs

Child module variables (modules/s3-bucket/variables.tf)

```
variable "bucket_name" {
  description = "Name of the S3 bucket"
  type        = string
}
variable "environment" {
  description = "Environment name"
  type        = string
}
```

Child module outputs (modules/s3-bucket/outputs.tf)

```
output "bucket_name" {
  description = "The name of the S3 bucket"
  value       = aws_s3_bucket.this.bucket
}
```

Key points

- Use variables to receive input values.
- Use outputs to return useful values to the root module (e.g. bucket name).
- Define clear descriptions and types for better usability.

7. Why module versioning matters

- Modules can be reused across multiple projects and environments.
- Versioning allows you to use a specific, tested version of a module.
- It prevents unexpected changes when the module is updated.
- You can use versioning when sourcing modules from a Git repository or the Terraform Registry.

Example: using a version from a Git repository

```
module "vpc" {
  source = "github.com/veriqta/terraform-aws-vpc?ref=v1.0.0"
  cidr_block = "10.0.0.0/16"
}
```

Key points

- `?ref=v1.0.0` uses a specific version tag.
- Helps keep your infrastructure consistent and stable.

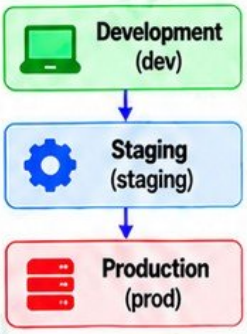
8. Key takeaway

- ✓ A module is a reusable collection of Terraform resources.
- ✓ Understand the difference between root and child modules.
- ✓ Use a clear folder structure.
- ✓ Pass input variables into a module and receive outputs.
- ✓ Use module versioning to keep your infrastructure stable and predictable.

Dev, Staging, and Production Environments

1. Why separate environments?

- Keeps development, testing and production isolated.
- Reduces the risk of accidental changes in production.
- Allows safe testing of new changes before they reach production.
- Helps with different configurations, cost controls and access permissions.
- Reflects real-world engineering practices.



2. Environment-specific configurations

- Each environment can have different values for things like instance sizes, counts, and features.
- You can use variables and .tfvars files to manage environment-specific settings.
- Example: smaller and cheaper resources in dev, larger and highly available resources in production.

dev.tfvars

```
environment = "dev"
instance_type = "t3.micro"
instance_count = 1
bucket_name = "veriqta-dev"
```

staging.tfvars

```
environment = "staging"
instance_type = "t3.small"
instance_count = 2
bucket_name = "veriqta-stg"
```

prod.tfvars

```
environment = "prod"
instance_type = "t3.medium"
instance_count = 3
bucket_name = "veriqta-prod"
```

3. Option 1: Separate root modules (recommended)

- Create a separate root module for each environment (dev, staging, prod).
- Each environment has its own configuration, state and backend.
- Provides clear separation and better control.

```
project/
├── dev/
│   ├── main.tf
│   ├── variables.tf
│   └── dev.tfvars
├── staging/
│   ├── main.tf
│   ├── variables.tf
│   └── staging.tfvars
└── prod/
    ├── main.tf
    ├── variables.tf
    └── prod.tfvars
```

Backend example for prod (S3)

```
terraform {
  backend "s3" {
    bucket  = "veriqta-terraform-prod"
    key     = "prod/terrafoom.tfstate"
    region  = "us-east-1"
    encrypt = true
    dynamodb_table = "veriqta-prod-locks"
  }
}
```

Key points

- ✓ Completely separate state files.
- ✓ Different credentials and permissions per environment.
- ✓ Best for production isolation.

4. Option 2: Terraform workspaces

- Workspaces allow you to use a single root module with multiple environments.
- Terraform keeps separate state files for each workspace.
- Useful for simple setups, but not enough for strong production isolation.

Workspace commands

```
$ terraform workspace list
  default
* dev
  staging
  prod

$ terraform workspace new staging
$ terraform workspace select prod
```

Key points

- Creates separate state files per workspace.
- Uses the same configuration code.
- Does not automatically enforce different credentials, permissions or access controls.
- Better for small teams or simple environments.

5. Why workspaces alone are not enough

- Workspaces only separate the state file, not your credentials, permissions or network access.
- The same IAM user/role might be used across all environments.
- A mistake in configuration can still affect the wrong environment.
- For production, use separate credentials, stricter IAM permissions and ideally separate root modules.

Workspaces are NOT a security boundary.
You still need proper access controls, separate credentials and environment-specific permissions.

6. Production awareness

Separate credentials	Different permissions	Separate state	Strong access controls
Use different AWS accounts or IAM roles for dev, staging and production.	Use least privilege access for each environment.	Store state files in different S3 buckets and use separate DynamoDB tables for locking.	Restrict who can make changes to production. Use approvals and proper processes.

7. Example: using variables for multiple environments

```
variable "environment" {
  description = "Environment name"
  type        = string
}

resource "aws_s3_bucket" "this" {
  bucket = "veriqta-${var.environment}-bucket"
  tags = {
    Environment = var.environment
}
```

Key points

- Use a variable for the environment name.
- Pass different values using dev.tfvars, staging.tfvars or prod.tfvars.
- Resources are named differently for each environment.

8. Key takeaway

- ✓ Separate environments reduce risk and improve stability.
- ✓ Use environment-specific configurations (variables and .tfvars).
- ✓ Separate root modules provide the strongest isolation.
- ✓ Workspaces can be useful but are not a security boundary.
- ✓ Always use separate credentials, permissions and state for production.
- ✓ Follow real-world engineering practices for environment management.

Git and Terraform: How Teams Work Together

1. Organize a Terraform repository

- A clear folder structure makes it easier for teams to work together.
- Keep your code organized by environments and modules.
- Include documentation (README) to help other team members understand the project.

```
terraform-project/
├── README.md       # Project documentation
├── main.tf         # Root module
├── variables.tf    # Input variables
├── outputs.tf      # Outputs
├── environments/   # Environment configurations
│   ├── dev/
│   ├── staging/
│   └── prod/
├── modules/        # Reusable modules
│   └── s3-bucket/
└── scripts/        # Helper scripts (optional)
```

2. Which Terraform files belong in Git?

- ✓ .tf files (main.tf, variables.tf, outputs.tf)
- ✓ Module directories (modules/)
- ✓ Environment directories (environments/)
- ✓ README.md and documentation
- ✓ .gitignore
- ✓ Example variable files (e.g. terraform.tfvars.example)
- ✓ Scripts and supporting files

Example: Important files to commit

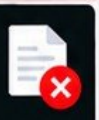
- main.tf
- variables.tf
- outputs.tf
- modules/
- environments/
- README.md
- .gitignore
- terraform.tfvars.example



3. Files that should be excluded from Git

- ✗ .terraform/ (provider plugins and local data)
- ✗ terraform.tfstate (state file)
- ✗ terraform.tfstate.backup
- ✗ *.tfvars (files with real values and secrets)
- ✗ .terraform.lock.hcl (can be committed, but usually optional)
- ✗ Any files containing sensitive information (API keys, passwords, secrets)

Example .gitignore



```
# Terraform local files
.terraform/
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.*

# Variable files with real values
*.tfvars
*.tfvars.json

# IDE / OS files
.vscode/
.DS_Store
```

4. How Git and pull requests work

- Developers make changes in a feature branch.
- A pull request (PR) is created to review the changes.
- Team members review the code, discuss and suggest improvements.
- After approval, the changes are merged into the main branch.



5. Review a Terraform plan before approving

- Always run terraform plan to see what changes will be made.
- Review the plan output in the pull request.
- Check for unexpected or destructive changes.
- Ensure the changes follow team standards and security policies.

Example: Plan output in a pull request

```
$ terraform plan

  # aws_s3_bucket.example will be created
+ resource "aws_s3_bucket" "example" {
    + bucket = "veriqta-demo-bucket"
    + tags = {
        + Environment = "dev"
      }
  }

Plan: 1 to add, 0 to change, 0 to destroy.
```

What to check

- ✓ Resources to be created, updated or destroyed
- ✓ Any unexpected changes
- ✓ Correct environment values
- ✓ Compliance with team standards
- ✓ No sensitive information in the plan output

6. Team collaboration best practices

- ✓ Use a consistent repository structure.
- ✓ Write clear commit messages.
- ✓ Use pull requests for all infrastructure changes.
- ✓ Review the terraform plan output before approval.
- ✓ Follow code review guidelines and team standards.
- ✓ Keep sensitive data out of Git.
- ✓ Use branches for new features or changes.
- ✓ Document your infrastructure so others can understand it.

7. Key takeaway

Git helps teams work together safely.	Keep the right files in Git.	Exclude local files, state files and secrets.	Use pull requests and review changes.	Always review the terraform plan before approving.

Terraform in CI/CD: From Code to Infrastructure

1. How Terraform fits into a CI/CD pipeline

- Automates infrastructure deployments using the same principles as application CI/CD.
- Detects errors early, ensures consistency and reduces manual work.
- Runs checks on every change before infrastructure is created or updated.
- Helps teams deliver infrastructure safely, quickly and repeatedly.

Terraform in CI/CD

2. CI/CD sequence: Format → Validate → Plan → Review → Apply → Verify

Format	Validate	Plan	Review	Apply	Verify
Check code style.	Check syntax and configuration.	Show what will change.	Team reviews the plan.	Create or update infrastructure.	Confirm resources exist and work as expected.

3. Example CI/CD flow (high level diagram)



4. Example: GitHub Actions workflow (simplified)

```yaml
name: Terraform CI/CD
on:
  pull_request:
    branches: [ main ]
jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3
    - name: Terraform Format
      run: terraform fmt -check
    - name: Terraform Validate
      run: terraform validate
    - name: Terraform Plan
      run: terraform plan -out=tfplan
    - name: Upload plan
      uses: actions/upload-artifact@v4
      with:
        name: tfplan
        path: tfplan
```

Key points

- Runs format, validate and plan on every pull request.
- Plan output is saved for review.
- Apply is only run after approval (e.g. on merge).
- Real workflows also include environment protection, secrets management and notifications.

5. Why production deployments need approval controls

- Reduces the risk of accidental or unauthorized changes.
- Allows team review of the plan before applying changes.
- Ensures changes follow security, compliance and business requirements.
- Helps prevent costly mistakes in production environments.

Never apply directly to production.
Always use a review and approval process.

Approval keeps production safe.

6. Short-lived credentials and workload identity

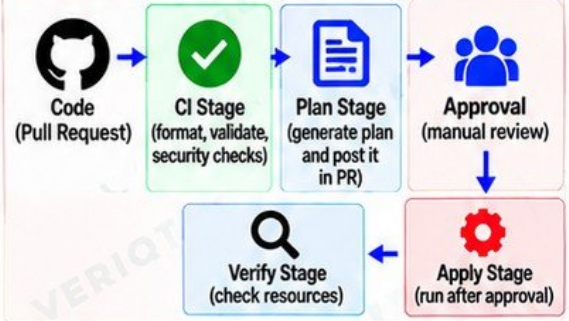
- Use short-lived credentials instead of long-lived access keys.
- GitHub Actions can use OIDC (OpenID Connect) to assume an IAM role in AWS.
- Credentials are generated for a short time and rotated automatically.
- This is more secure than storing permanent access keys.



Key points

- No hardcoded access keys.
- Credentials are temporary.
- Use IAM roles with least privilege access.
- Works with AWS and other cloud providers.

7. Full pipeline example (multi-stage)



8. Key files and tools in CI/CD

File / Tool	Purpose
.github/workflows/*.yml	Defines the CI/CD pipeline.
`terraform fmt`	Checks and fixes code formatting.
`terraform validate`	Validates configuration syntax.
`terraform plan`	Shows what will change.
`terraform apply`	Creates or updates infrastructure (after approval).
OIDC / IAM role	Provides short-lived cloud credentials.

9. Environment-specific pipelines

- Use separate pipelines or jobs for dev, staging and production.
- Use environment protection rules for production (e.g. required reviewers).
- Use separate credentials and state for each environment.
- Apply automatically for dev/staging if allowed, but always require approval for production.

Dev (auto apply)
Staging (auto or manual)
Production (manual approval only)

10. Key takeaway

- Terraform can be integrated into a CI/CD pipeline.
- Automate format, validate, plan, review, apply and verify.
- Use approval controls for production.
- Use short-lived credentials and workload identity.
- Keep environments separate with proper permissions and state.
- A well-designed pipeline makes infrastructure changes safer, faster and more reliable.

Terraform Errors: Learn to Read the Evidence

1. Common Terraform errors and what they mean

- Errors are normal when working with Terraform.
- The error message helps you understand what went wrong.
- Read the full message, identify the cause, and follow the hints.
- Use terraform validate and terraform plan to catch issues early.
- Most errors fall into a few common categories.



2. Invalid configuration errors

These happen when your Terraform code has syntax errors or missing required arguments.

```
$ terraform validate

Error: Missing required argument
on main.tf line 3, in resource "aws_s3_bucket" "example":
  3: resource "aws_s3_bucket" "example" {

The argument "bucket" is required,
but no definition was found.
```

How to fix

- Check the error line number.
- Add the missing argument.
- Use terraform validate to find syntax issues.

3. Missing provider errors

These happen when the required provider is not installed or not defined in your configuration.

```
Error: Invalid provider configuration

Provider "registry.terraform.io/hashicorp/aws"
was not found in the configured providers.

To install this provider, run:
  terraform init
```

How to fix

- Add the provider in required_providers.
- Run terraform init to download providers.
- Check your internet connection and provider version.

4. Authentication and permission errors

These happen when Terraform cannot authenticate with your cloud provider or does not have enough permissions.

```
Error: error configuring Terraform AWS Provider:

no valid credential sources for AWS Provider
found.

Error: AccessDenied: User: arn:aws:iam::123456789012
is not authorized to perform: s3:CreateBucket
on resource: arn:aws:s3:::veriqta-demo-bucket
```

How to fix

- Check your AWS credentials are configured correctly.
- Ensure the IAM user/role has the required permissions.
- Use least privilege access.
- Verify you are using the correct AWS account and region.

5. Resource naming conflicts

These happen when a resource with the same name already exists (e.g. S3 bucket names must be globally unique).

```
Error: creating S3 Bucket (veriqta-demo-bucket):

BucketAlreadyExists:
The requested bucket name is not available.

StatusCode: 409, RequestID: 1234567890
```

How to fix

- Use a unique name (e.g. include a prefix, random string or your company name).
- Check if the resource already exists.
- Follow your naming conventions.

6. Unavailable region or service errors

These happen when the resource or service is not available in the selected region.

```
Error: Unsupported region

The requested region "us-west-3" is not enabled
or is not supported by the AWS provider.

Valid regions include: us-east-1, us-west-1,
us-west-2, eu-west-1 ...
```

How to fix

- Check the correct region name.
- Ensure the service is available in that region.
- Use a supported region for your resources.

7. Useful Terraform commands for troubleshooting

`terraform validate`	Checks your configuration for syntax errors.
`terraform init`	Initializes the working directory and downloads providers.
`terraform plan`	Shows what changes will be made and surface any errors.
`terraform providers`	Shows the providers being used.
`terraform version`	Checks your Terraform version.
`TF_LOG=DEBUG terraform plan`	Shows detailed logs for deeper troubleshooting.

8. Practice: Find and fix the mistake

The following configuration has a mistake. Identify the error, read the message and fix it.

Broken configuration (main.tf)

```
resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"
  region = "us-east-1"  # X invalid
}
```

Corrected configuration

```
resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"
  # region is not a valid argument here
  # (region is set in the provider)
}
```

9. Troubleshooting checklist

- ✓ Read the full error message.
- ✓ Check the line number and resource.
- ✓ Verify your configuration syntax.
- ✓ Ensure providers are installed and initialized.
- ✓ Check your credentials and permissions.
- ✓ Confirm the resource name is unique.
- ✓ Verify you are using a supported region.
- ✓ Use terraform validate and terraform plan.
- ✓ Check detailed logs if needed (TF_LOG=DEBUG).
- ✓ Make the fix and run the command again.

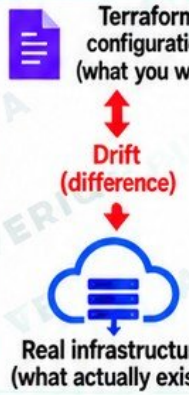
10. Key takeaway

- Terraform error messages give you clear clues.
- Most errors are caused by configuration, authentication, permissions, naming or region issues.
- Use the Terraform commands and logs to investigate.
- Read, understand, fix and try again.
- Troubleshooting is a key skill for real-world DevOps work.

Infrastructure Drift: When Reality Changes

1. What is infrastructure drift?

- Drift is a difference between your Terraform configuration and the real infrastructure.
- It happens when a resource is changed outside of Terraform (e.g. manually in the cloud console, by another tool or by another team).
- Terraform still thinks the resource matches your code, but the real resource is now different.



2. Example: someone changes a resource manually

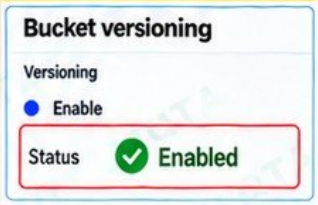
An engineer changes the S3 bucket versioning setting in the AWS console. Terraform configuration has versioning = false, but the real bucket now has versioning = true.

Terraform configuration (main.tf)

```
resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"

  versioning {
    enabled = false
  }
}
```

Manual change in AWS console (outside Terraform)



3. Detecting drift with terraform plan

Run terraform plan to compare your configuration with the real infrastructure.

```
$ terraform plan
# aws_s3_bucket.example will be updated in-place
~ resource "aws_s3_bucket" "example" {
      id      = "veriqta-demo-bucket"
    ~ versioning {
        ~ enabled = true -> false
      }
  }

Plan: 0 to add, 1 to change, 0 to destroy.
```

Key points

- Terraform shows the difference between what is in your code and what exists in the cloud.
- It does not mean something is wrong, but it needs your attention.
- Investigate why the change happened before applying.

4. Using terraform plan -refresh-only

The -refresh-only option updates the state file with the real infrastructure but does not propose any changes to make.

```
$ terraform plan -refresh-only

Refreshing state...

No changes. Your infrastructure
matches the configuration.
```

Key points

- Updates the state with the current real-world values.
- Does not show changes to apply.
- Useful to confirm what is actually in the cloud.
- It does not fix drift, it only refreshes state.

5. Common causes of drift

Manual changes in the cloud console
Someone edits a resource directly in AWS, Azure or GCP.

Changes by other tools
Another tool or script modifies the resource.

Changes by another team
A different team makes a change without updating the Terraform code.

Cloud provider updates
Some services make automatic changes (e.g. default settings, maintenance).

6. Investigate before you apply

1. Review the plan output and understand the changes.
2. Check who made the change and why.
3. Verify in the cloud console or with CLI commands.
4. Decide if the change should be kept or reverted.
5. Only apply when you are sure of the correct action.

Never blindly apply a plan.
An unexpected change could be important, and applying might remove a required manual fix or cause an outage.

7. Useful commands for investigating drift

Command	What it does
`terraform plan`	Shows differences between your code and real infrastructure.
`terraform plan -refresh-only`	Refreshes the state with real values (no changes).
`terraform state show <resource>`	Shows the current state for a specific resource.
`terraform state list`	Lists all resources in your state file.
`aws s3api get-bucket-versioning --bucket <name>`	Check the real setting in AWS (example for S3).

8. Key takeaway

- Drift is a difference between your configuration and reality.
- It can happen due to manual changes, other tools, teams or cloud provider updates.
- Use terraform plan to detect drift and investigate.
- terraform plan -refresh-only updates the state but does not fix drift.
- Always understand the cause before applying changes.
- Prevent drift with proper access controls, team processes and monitoring.

Dangerous Terraform Mistakes and Safe Recovery

1. terraform destroy vs terraform state rm

These commands do very different things and are often confused.

terraform destroy

- Destroys real infrastructure resources.
- Permanently deletes resources in your cloud environment.
- Use with extreme caution.

```
$ terraform destroy

Plan: 3 to destroy.

Do you really want to destroy
all resources?
  Enter a value:  yes
```

terraform state rm

- Removes a resource from the Terraform state file.
- Does NOT destroy the real resource in the cloud.
- Used when Terraform is no longer managing an existing resource.

```
$ terraform state rm
    aws_s3_bucket.example

Removed aws_s3_bucket.example
from state.
```

2. Resource replacement can cause downtime or data loss

- Some changes require Terraform to replace a resource instead of updating it.
- Replacement can cause downtime, data loss or service interruption.
- Common examples: changing certain immutable fields, modifying storage types, or renaming resources.

Example: resource replacement in plan

```
# aws_db_instance.example must be replaced
-/+ resource "aws_db_instance" "example" {
    ~ instance_class = "db.t3.micro" -> "db.t3.small" # forces replacement
      # (other attributes unchanged)
  }
Plan: 1 to add, 1 to destroy.
```

Replacement may destroy and recreate the resource.
Always review the plan and understand the impact before applying.

3. prevent_destroy and its limitations

- prevent_destroy helps protect important resources from being accidentally destroyed.
- It blocks destroy and some replacement actions.
- It does not protect against all types of changes (e.g. manual deletion, state removal or changes outside Terraform).

Example: prevent_destroy configuration

```
resource "aws_s3_bucket" "important" {
  bucket = "veriqta-important-bucket"

  lifecycle {
    prevent_destroy = true
  }
}
```

If you try to destroy

```
Error: Instance
cannot be destroyed

The resource
aws_s3_bucket.important
has lifecycle.prevent_
destroy set to true.
```

4. Manual state editing is dangerous

- The state file tracks what Terraform manages.
- Manually editing the state file can corrupt it.
- It can cause unexpected changes, resource recreation or loss of resources from management.
- Only edit state manually in rare cases and with expert knowledge.

Do not edit terraform.tfstate manually.
Use Terraform commands (e.g. terraform state rm, terraform import) instead.

5. Common dangerous mistakes

- Running terraform destroy in the wrong directory.
- Applying a plan without reviewing the changes.
- Changing immutable fields that force replacement.
- Removing resources from state without understanding the impact.
- Using the wrong variables, workspace or environment.

6. Safe response: Stop → Inspect → Review State → Plan → Approve → Verify

1. **Stop** — Do not panic and do not apply changes immediately.
2. **Inspect** — Understand what changed and why.
3. **Review State** — Check the current state and compare with reality.
4. **Plan** — Run terraform plan and carefully review the output.
5. **Approve** — Get the right approval, especially for production changes.
6. **Verify** — After applying, confirm the resources are in the expected state and working.

7. Useful commands for recovery and investigation

Command	Purpose
terraform plan	See what changes will be made.
terraform plan -refresh-only	Check for drift without proposing changes.
terraform state list	List all resources in the state.
terraform state show <resource>	Show details of a resource in the state.
terraform import <address> <id>	Import an existing resource into state.
terraform state rm <address>	Remove a resource from state (does not destroy it).

8. Key takeaway

- Understand the difference between terraform destroy and terraform state rm.
- Resource replacement can cause downtime or data loss.
- Use prevent_destroy for important resources, but know its limits.
- Manual state editing is dangerous.
- Always follow the safe response: Stop → Inspect → Review State → Plan → Approve → Verify.
- Careful planning and review prevent costly mistakes.

Your First Complete Terraform Infrastructure Project

1. Project overview

- Build a small AWS infrastructure project using the concepts learned.
- Create an S3 bucket with secure public access settings.
- Use input variables, locals, resource references and outputs.
- Initialize, validate, plan, apply and verify the deployment.
- Make one configuration change and inspect the plan.
- Finally, destroy the infrastructure in the dedicated practice environment.



2. Project structure

```
terraform-aws-s3-project/
├── main.tf          # S3 bucket and settings
├── variables.tf     # Input variables
├── locals.tf        # Local values
├── outputs.tf       # Output values
├── provider.tf      # AWS provider configuration
├── versions.tf      # Terraform and provider versions
└── README.md        # Project documentation
```

3. Key Terraform configuration files (examples)

provider.tf

```
provider "aws" {
  region = var.aws_region
}
```

variables.tf

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "bucket_name" {
  type        = string
  description = "Name for the S3 bucket"
}
```

main.tf (S3 bucket with secure settings)

```
resource "aws_s3_bucket" "example" {
  bucket = var.bucket_name
}

resource "aws_s3_bucket_public_access_block" "example" {
  bucket                  = aws_s3_bucket.example.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

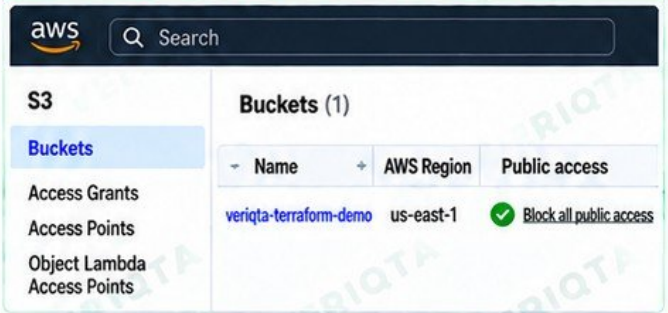
outputs.tf

```
output "bucket_name" {
  value = aws_s3_bucket.example.bucket
}
```

4. Terraform workflow: Initialize → Verify

1	Initialize	`terraform init` — Downloads required providers.
2	Validate	`terraform validate` — Checks configuration syntax.
3	Plan	`terraform plan` — Shows what will be created.
4	Apply	`terraform apply` — Creates the infrastructure.
5	Verify	Check in AWS console — Confirm the S3 bucket exists with secure public access settings.

5. Verify the deployment in AWS



6. Make a configuration change and inspect the plan

- Example: add a tag to the bucket.
- Run `terraform plan` to see the changes.
- Verify that only the intended changes will be made.

```
# Added a tag in main.tf
tags = {
  Project = "veriqta-terraform"
  Environment = "practice"
}
```

Plan output (summary)

```
~ resource "aws_s3_bucket" "example" {
    tags = {
      + Environment = "practice"
      + Project     = "veriqta-terraform"
    }
}
Plan: 0 to add, 1 to change, 0 to destroy.
```

7. Destroy the infrastructure (practice environment only)

- Run `terraform destroy` in a dedicated practice environment.
- Confirm that the resources are removed.

```
$ terraform destroy

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
  Enter a value: yes

aws_s3_bucket.example: Destroying...
aws_s3_bucket.example: Destruction complete!
```

Only destroy in your practice environment.

Never destroy resources in production or shared environments.

8. Final engineering lesson

- Infrastructure is not finished until it is verified, secured, documented and managed safely.
- Always review the plan output before applying.
- Use secure configurations (e.g. block public access).
- Keep your code organized with variables, locals and outputs.
- Test changes in a safe environment.
- Clean up resources when you no longer need them.
- Follow team standards, access controls and monitoring.
- Think like an engineer: plan, verify and manage, not just deploy.